

Stockbot / Capital Ledger

Automates trading Indian equities on SPTulsian advisory calls, via Zerodha Kite, with budget-aware position sizing and a full ledger of what it did.

 [ARCHITECTURE.md](#) (PDF) — functional and technical design, failure handling, and the reasoning behind the non-obvious parts. Read that before changing anything load-bearing.

This README is also available as a [PDF](#). Both are committed artefacts — regenerate them with `./venv/bin/python tools/md2pdf.py --all` after editing either source, or they go stale silently. (The venv interpreter, not bare `python3`: the converter needs `markdown`, which is installed there.)

How it runs

Seven scheduled jobs plus three maintenance loops. All cron times are **UTC**; the VM clock is UTC and IST is UTC+5:30. The canonical schedule is `provisioning/crontab` — edit that, not `crontab -e`, then run `bash provisioning/install_crontab.sh`.

Job	UTC	IST	What it does
<code>main_recommend.py</code>	<code>0 4</code>	09:30	Parse the advisory email, resolve symbols, scrape targets, backfill held trades. No orders.
<code>spt_selfheal.py</code>	<code>30 4</code>	10:00	If the 09:30 scrape failed: repair the tunnel and re-run — while the buy can still happen
<code>main_conviction.py</code>	<code>45 4</code>	10:15	Score new recommendations. Sets position size — a failure here holds buying
<code>spt_watchdog.py</code>	<code>15 5</code>	10:45	Pass 1 — are the buy inputs ready?
<code>main.py</code>	<code>30 5</code>	11:00	Price, size against budget, place real buy orders
<code>spt_watchdog.py</code>	<code>45 5</code>	11:15	Pass 2 — did the buy actually happen ?
<code>main_gtt_oracle.py</code>	<code>30 10</code>	16:00	Place GTT sells; close trades on confirmed fills

And three that run all day:

Loop	Cron	What it does
<code>provisioning/warp_keeper.sh</code>	<code>*/20</code>	Restarts <code>warp-svc</code> when a real fetch fails, or at 180 MB — before it degrades
<code>provisioning/sample_warp_memory.sh</code>	<code>*/30</code>	Records memory; this is where the ~28 MB/day leak rate comes from
DuckDNS refresh	<code>*/5</code>	Dynamic DNS

Every repair runs **before** the thing it protects — self-heal at 10:00 ahead of scoring at 10:15 ahead of buying at 11:00. A repair scheduled after the buy window would be a report, not a recovery.

The watchdog runs twice for a reason. Everything before 11:00 only verifies the buy's *inputs*; none of it can see a run that had good inputs and then failed while executing. A run that dies mid-flight still logs "Buy Phase complete", which is how two days of buying were lost on 2026-09-01/02.

The 90-minute gap between recommend and buy is deliberate: it is the window in which a human can fix a mis-resolved ticker **before** money moves. A symbol the bot cannot resolve confidently becomes `NEEDS_REVIEW` and is never bought on a guess.

Run on demand:

```
python3 main_conviction.py --dry-run      # score today's, print, write nothing
python3 main_conviction.py --engine full  # the deeper fundamentals engine
python3 main_conviction.py --all-open     # re-score every open position
python3 tools/dryrun_sizing.py            # what the buy run WOULD do; writes nothing
python3 tools/recalibrate_bands.py        # re-derive sizing cutoffs
python3 spt_capture.py --dry-run          # preview scraped targets, then save
python3 spt_watchdog.py --check-only      # health check, never sends mail
python3 spt_selfheal.py --check-only      # what would it repair? changes nothing
bash provisioning/warp_keeper.sh --check  # real fetch through the proxy + memory
bash provisioning/install_crontab.sh --check # has the live schedule drifted?
```

Run `tools/dryrun_sizing.py` before any change that affects money — it mirrors the real buy path against live data with every write and order call replaced by a tripwire that raises.

Layout

Only entrypoints live at the repository root — one file per thing you can run. Everything they share is in `lib/`, and superseded scripts are in `archive/`.

```

.
├─ main_recommend.py      # Phase 1: symbols + targets, no orders      ← cron
├─ main.py                # Phase 2: price, size, buy                ← cron
├─ main_gtt_oracle.py     # Phase 3: GTT sells, confirm fills, close ← cron
├─ spt_watchdog.py        # Liveness alarm for the scraper                ← cron
├─ spt_selfheal.py        # Repair the scrape path and re-run the day ← cron
├─ main_conviction.py     # Evidence scoring; the lite score sets position size
├─ spt_capture.py         # Manual review-then-save of scraped targets
├─
├─ lib/                   # shared modules
│   ├─ config.py          # env, credentials, KITE_ACCOUNT switch
│   ├─ kite_client.py     # Kite auth, symbol resolution, orders, GTTs
│   ├─ order_status.py    # order lookup, sell reconciliation
│   ├─ email_reader.py    # Gmail IMAP -> tips
│   ├─ budget_manager.py  # budget policy; all TRADES reads/writes
│   ├─ spt_scraper.py     # portal login + two parsing strategies
│   ├─ conviction.py      # four-layer fundamentals engine (--engine full)
│   ├─ conviction_lite.py # reachability/momentum/trend/liquidity (default)
│   ├─ bands.py           # sizing thresholds - ONE definition, read by bot + UI
│   └─ sheet_logger.py    # Google Sheets mirror (legacy)
├─
├─ dashboard/             # Streamlit UI ("Capital Ledger")
│   ├─ app.py             # pages
│   ├─ db.py              # Oracle access
│   ├─ kite_data.py       # multi-account sync, symbol resolution (places NO orders)
│   ├─ capture_api.py     # authenticated endpoint for spt_capture.py
│   └─ theme.py           # CSS design system
├─
├─ migrations/            # numbered, idempotent Oracle DDL
├─ tools/                 # recalibrate_bands.py, dryrun_sizing.py, md2pdf.py
├─ archive/               # superseded / one-off - see archive/README.md
├─ bot/                   # multi-tenant variant - NOT scheduled
├─ provisioning/          # crontab (canonical), systemd drop-ins, tenant onboarding
│   ├─ crontab            # the schedule - edit this, never `crontab -e`
│   ├─ install_crontab.sh # installs it; --check reports drift
│   ├─ warp_keeper.sh     # recycles warp-svc before it degrades      ← cron
│   └─ sample_warp_memory.sh # leak-rate evidence                    ← cron
└─ tests/                 # logic validated against SQLite mocks

```

Entrypoints stay at the root deliberately: cron invokes them by bare filename (`cd /home/ubuntu/stock_bot_v4 && python3 main_recommend.py`), so moving them would mean editing the crontab on a live system for no benefit.

`bot/` is a multi-tenant version (schema-per-tenant, encrypted per-tenant credentials) that is not in the live crontab and keeps its own module copies — it does **not** import from `lib/`. The scheduled system is the single-tenant root tree. Keep the two in sync when changing shared logic, or they will silently diverge.

Deploy

Two directories on the VM track the same repo: `/home/ubuntu/stock_bot_v4` (cron) and `/home/ubuntu/stockbot` (dashboard).

```
git add -A && git commit && git push
ssh -i ~/.ssh/kite_key ubuntu@140.245.226.35 '
  cd /home/ubuntu/stock_bot_v4 && git pull -q
  cd /home/ubuntu/stockbot      && git pull -q
  sudo systemctl restart stockbot-dashboard
  cd /home/ubuntu/stock_bot_v4 && git status --porcelain' # MUST print nothing
```

git pull is the only supported deploy. `scp` also works, which is the trap: the VM then runs code that exists nowhere else, and a dirty `git status` is the only evidence. That happened for ten days in September 2026 — a blocked push, deploys carried on by hand, and no way to answer "is production running the fix?" without checksumming files one by one. Nothing broke; the loss was knowing. See [ARCHITECTURE.md §5.1](#).

Health check

```
ssh -i ~/.ssh/kite_key ubuntu@140.245.226.35 '
  curl -s ifconfig.me; echo # MUST be 140.245.226.35
  curl -s --socks5-hostname 127.0.0.1:40000 ifconfig.me; echo # MUST be Cloudflare
  sudo systemctl is-active warp-svc stockbot-dashboard stockbot-capture-api
  free -m # swap MUST be non-zero
  cd /home/ubuntu/stock_bot_v4
  bash provisioning/warp_keeper.sh --check # a REAL fetch + memory; see below
  bash provisioning/install_crontab.sh --check # schedule drift
  python3 spt_selfheal.py --check-only # would it need to repair anything?
  python3 spt_watchdog.py --check-only # never sends mail
  git status --porcelain' # MUST be empty
```

The VM has 956 MB of RAM and that is the binding constraint. `warp-svc` leaks and took the box down on 2026-08-26 by triggering a global OOM that killed `sshd` and Caddy — from outside, ports 22 and 443 still completed a TCP handshake but nothing ever replied. It is now capped at `MemoryMax=300M` with `Restart=always`, plus a 2 GB swapfile and a 200 MB journald cap. If SSH ever hangs at "banner exchange" again, that is this, and it needs a **force** reboot from the OCI console — a graceful one hangs, because `systemd` is part of what is stuck. See [ARCHITECTURE.md §4.4](#).

The cap alone was not enough, because `warp-svc` degrades before it dies. At 230 MB — under the cap, so never restarted — it reported "Connected / Network healthy" while the proxy returned HTTP 000 to everything. `warp-cli connect` did nothing; a restart fixed it in three seconds.

So **never trust `systemctl is-active` or `warp-cli status` here.** They report that a process exists and what the tunnel believes; neither answers whether the scraper can fetch. `warp_keeper.sh` asks the only question that matters by making a real request, and recycles at 180 MB — inside the degradation window, below the cap. At ~28 MB/day that is roughly twice a week.

If the first command ever returns a Cloudflare address, stop. The scraper's proxy has escaped its scope and broker traffic is no longer leaving from the IP Zerodha expects. See [ARCHITECTURE.md §3.2](#).

Network egress

`sptulsian.com` blocks datacenter IPs at the CloudFront edge, so the VM cannot reach it directly. Scraper traffic — and only scraper traffic — goes through a Cloudflare WARP SOCKS5 proxy on `127.0.0.1:40000`, attached to a single `requests.Session`. Broker traffic must keep egressing from the registered static IP, because Zerodha binds to it.

The env var is deliberately named `SPTULSIAN_PROXY` rather than `HTTP_PROXY` / `HTTPS_PROXY`: `requests` honours the standard names automatically for every module in the process, which would silently route order placement through the proxy.

`.env`

Lives at `/home/ubuntu/.env` on the VM, `export KEY=value` format. Never in the repo.

```
# Kite / broker
KITE_API_KEY=          KITE_API_SECRET=
ZERODHA_USER_ID=       ZERODHA_PASSWORD=       ZERODHA_TOTP_SECRET=
ZERODHA_OLD_USER_ID=   ZERODHA_OLD_PASSWORD=   ZERODHA_OLD_TOTP_SECRET=

# Advisory
GMAIL_USER=           GMAIL_APP_PASSWORD=
SPT_USERNAME=         SPT_PASSWORD=
SPTULSIAN_PROXY=socks5h://127.0.0.1:40000

# Oracle
ORACLE_USER=          ORACLE_PASSWORD=          ORACLE_DSN=
ORACLE_WALLET_DIR=    ORACLE_WALLET_PASSWORD=

# Dashboard + capture API
DASH_USERS=user:pass,...
DASH_SESSION_SECRET=  # 64-char hex; signs the "remember me" cookie
CAPTURE_API_KEY=      CAPTURE_API_URL=
```

`KITE_ACCOUNT=OLD` on a one-off run targets the pre-cutover personal Zerodha account instead of the automation account. Cron never sets it, so scheduled jobs always use the new account.

```
KITE_ACCOUNT=OLD python3 main_gtt_oracle.py
```

Working on this safely

The invariants below hold by construction. Preserve them.

1. Broker traffic egresses from the registered static IP. Always.
2. A symbol that does not resolve cleanly is never bought — it becomes `NEEDS_REVIEW` and waits for a human.
3. A trade is `Closed` only on a **confirmed** sell fill, never on GTT status alone.
4. Conviction sizing is **on** (`CONVICTION_SIZING_ENABLED`), using the lite engine: `>92 → ₹25,000` , `69–92 → ₹10,000` , below 69 not bought. The thresholds are percentile-matched to the lite score distribution, not carried over from the full engine — see `lib/bands.py` , which is their single definition and the one the dashboard reads too. A score can therefore now *skip* a recommendation, so a conviction-run failure holds buying; `spt_watchdog.py` alarms on unscored pending buys. Note the score is still **unvalidated**: the only backtest to date found no relationship between conviction and excess return. Re-run `python3 backtest_conviction.py` on `model='lite'` rows as trades close, and set the flag `False` to return to flat `INVEST_AMT` .
5. **The dashboard never places an order.** Every buy goes through `main.py` , so there is one code path that spends money and one place to gate it. Needs Review confirms *which instrument* a tip is; it does not buy.
6. A quantity is never claimed from a symbol-level lookup. Holdings are per symbol, positions are per lot — anything inferring "this order filled" from a holding must subtract what other open lots already claim.
7. An explicit SELL from SPTulsian is never bought, and always reaches a human (`ADVISORY_SELL` + watchdog mail).
8. Thresholds and status lists are defined once — `lib/bands.py` for sizing cutoffs, `NON_BUYING_STATUSES` for statuses meaning no stock was bought. Every duplicated constant here has eventually drifted.

Anything touching money or the ledger gets exercised first in an isolated directory with Oracle writes, Sheet writes, and order placement monkey-patched out, run against real live data and inspected before the real run. That convention has caught a pooled-budget bug, a GTT close that recorded no sell price, and a `DataFrame.__bool__` call that silently nulled every financial statement in the conviction engine.

Three failures worth knowing before you touch reconciliation, the scraper, or the retry path — documented in [ARCHITECTURE.md §4.5–4.6](#):

- A holdings-based fill inference claimed all 8 BSE shares for a 3-share order that never filled, recording ₹26,608 of stock that was never bought.
- A SELL call was dropped at a regex that hardcoded `(Buy @ price)` — not bought, but never recorded or surfaced either.
- A trade whose scrape failed was retried daily for its **buy**, but nothing ever retried its **lookup**, so the missing data could never arrive. Four trades looped that way for two days before ageing out unbought.

Note on the conviction layer

Two engines. **lite** is the default and sizes real positions. Four cheap, always-available components, one network call per symbol:

Component	Points
Reachability — $\text{gap} / (\text{vol} \times \sqrt{63})$	40
Momentum 12-1	25
Trend alignment	20
Liquidity	15

`--engine full` is the original four-layer fundamentals engine, kept for a deep look at one name. Nothing sizes on it. Their scores are **not comparable** and are stored with a `model` column; never pool them in a backtest, and note the dashboard badge reads that column so a full-engine score is never coloured by lite thresholds.

Why reachability replaced "upside to target". SPTulsian sets targets at a near-constant ~6% above their recommended price — across 16 closed trades the gap ran 5.82% to 7.69%, sd 0.58pp. Scoring that across a 0–30% range gave every stock almost the same value on a quarter of the composite. What actually separates a name that reaches target in 3 days from one that takes 23 is volatility, not distance: $\text{gap} / (\text{vol} \times \sqrt{63})$ ranked time-to-target at rho +0.57 against +0.36 for the raw gap.

Read the caveats before trusting the number. Those 16 trades are all winners — positions close when they touch target, and ones that fail just stay open — so the model is fitted to the easy half of the distribution. And optimising for speed-to-target favours volatility, which with no stop-loss makes losing positions worse. Hit rate and tail risk move together here, and only the first is currently measurable. That trade-off was chosen deliberately.

`reach_z` is stored per score so the honest test — stored z against realised time-to-target — becomes possible once enough of the book resolves.

The thresholds are derived, not constants. They have been recalibrated three times as components changed. Run `tools/recalibrate_bands.py` after any change to the engine, or the numbers survive while the policy silently shifts.

It computes published metrics from public data and shows its working. It is decision support, not financial advice, and not a prediction — every threshold in it is a convention tuned against one portfolio.